SWYNEX TECHNOLOGIES

SECURITY ASSESSMENT REPORT

Web Application Security Assessment

OWASP Top 10 Security Review

OWASP Juice Shop

SUBMITTED TO
SWYNEX Technologies

SUBMITTED BY
Fathima Ifadha
BSc Cybersecurity & Digital Forensics Undergraduate

INTERNSHIP PROGRAM
SWYNEX Cybersecurity Internship

ASSESSMENT TARGET
OWASP Juice Shop

ASSESSMENT ENVIRONMENT
Authorized Local Testing Environment

REPORT DATE
November 2026

PROJECT TYPE
Defensive Web Application Security Assessment

TABLE OF CONTENTS

1. Executive Summary
2. Introduction
3. Objectives
4. Scope of Assessment
5. Methodology
6. Tools Used
7. Security Assessment Findings
    - A01: Broken Access Control
    - A02: Security Misconfiguration
    - A03: Software Supply Chain Failures
    - A04: Cryptographic Failures
    - A05: Injection
    - A06: Insecure Design
    - A07: Authentication Failures
    - A08: Software or Data Integrity Failures
    - A09: Security Logging and Alerting Failures
    - A10: Mishandling of Exceptional Conditions
8. Summary of Risk Ratings
9. Recommendations
10. Limitations
11. Conclusion

1. Executive Summary

This report presents the security assessment of the OWASP Juice Shop web application. The assessment was performed as part of the SWYNEX Cybersecurity Internship.

The main purpose of this assessment was to identify possible security weaknesses by reviewing the OWASP Top 10 categories.

The testing was performed in an authorized local lab environment. Tools such as Burp Suite Community Edition, web browser developer tools, Docker Desktop, and PowerShell were used during the assessment.

During the assessment, several security issues and potential weaknesses were identified. These included unauthorized access to user information, IDOR, public access to sensitive files, SQL error messages, and session handling issues.

Some findings were confirmed through testing, while others could not be fully verified.

This report explains the findings, their risk levels, and possible recommendations to improve application security.

2. Introduction

Web application security is important because web applications store and process different types of user information.

Security weaknesses can allow attackers to access private information, change data, or affect application availability.

For this assessment, OWASP Juice Shop was used as an intentionally vulnerable web application. It provided a safe environment to understand and test common web security issues.

The assessment focused on the OWASP Top 10 categories.

3. Objectives

The main objectives of this assessment were:

- To understand common web application security risks.
- To review the OWASP Top 10 categories.
- To identify security weaknesses in the test application.
- To perform basic security testing using available tools.
- To collect evidence for confirmed findings.

- To provide recommendations to improve application security.
- To improve practical cybersecurity knowledge.

4. Scope of Assessment

4.1 Target Application

OWASP Juice Shop was used as the target application.

4.2 Testing Environment

The assessment was conducted in a local testing environment using Docker Desktop.

4.3 Assessment Scope

The assessment included:

- Access control testing.
- Security configuration review.
- Software and dependency exposure checks.
- Basic cryptographic security observations.
- SQL injection and input handling checks.
- Application design observations.
- Authentication and session management testing.
- Software and data integrity observations.
- Security logging review.
- Error handling and application stability checks.

4.4 Authorization

All testing was performed on the intentionally vulnerable local lab application.

No unauthorized external systems were tested.

5. Methodology

The assessment was completed using a manual testing approach.

The following steps were followed:

1. Set up the OWASP Juice Shop application in a local environment.
2. Opened the application using a web browser.

3. Reviewed application requests and responses.
4. Used Burp Suite to inspect HTTP traffic.
5. Tested selected application features and inputs.
6. Reviewed access control and authentication behaviour.
7. Checked exposed files and application error messages.
8. Collected screenshots for important findings.
9. Recorded the results and prepared recommendations.

The testing focused on the OWASP Top 10 categories.

6. Tools Used

The following tools were used during the assessment:

1. OWASP Juice Shop

Used as the intentionally vulnerable web application.

2. Burp Suite Community Edition

Used to inspect HTTP requests and responses.

3. Web Browser

Used to access the application and perform basic testing.

4. Browser Developer Tools

Used to inspect application behaviour and browser information.

5. Docker Desktop

Used to run the OWASP Juice Shop application locally.

6. PowerShell

Used to manage and check the local testing environment.

---

7. SECURITY ASSESSMENT FINDINGS

7.1 A01 – Broken Access Control

Risk Level: Medium

During testing, some access control weaknesses were identified.

The application allowed access to another user's basket information by changing the basket identifier. This showed an IDOR (Insecure Direct Object Reference) issue.

The `/api/Users` endpoint also returned information about multiple users, including email addresses and roles.

The admin page returned a 403 Forbidden response during the tested scenario. Therefore, unauthorized admin access was not confirmed.

SSRF was also not confirmed during this assessment.

Finding 1: IDOR – Unauthorized Basket Access

Status: Confirmed
Risk Level: Medium

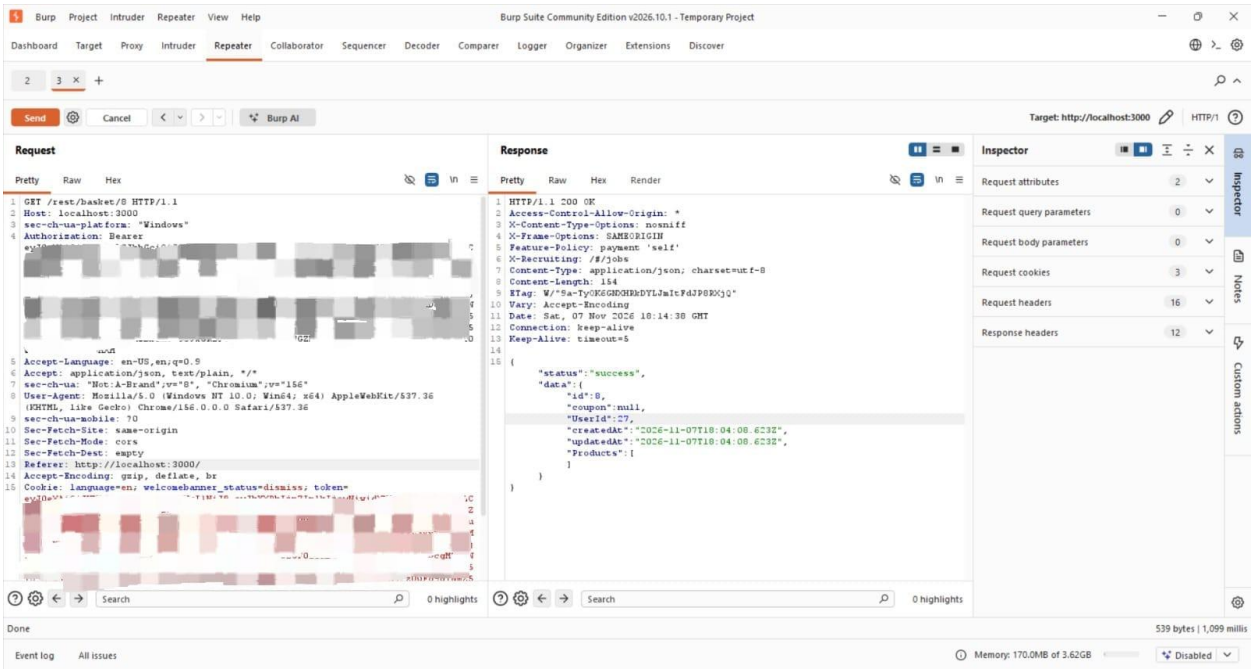


Figure 1: IDOR – Accessing another user's basket.

Finding 2: User Information Exposure

Status: Confirmed
Risk Level: Medium

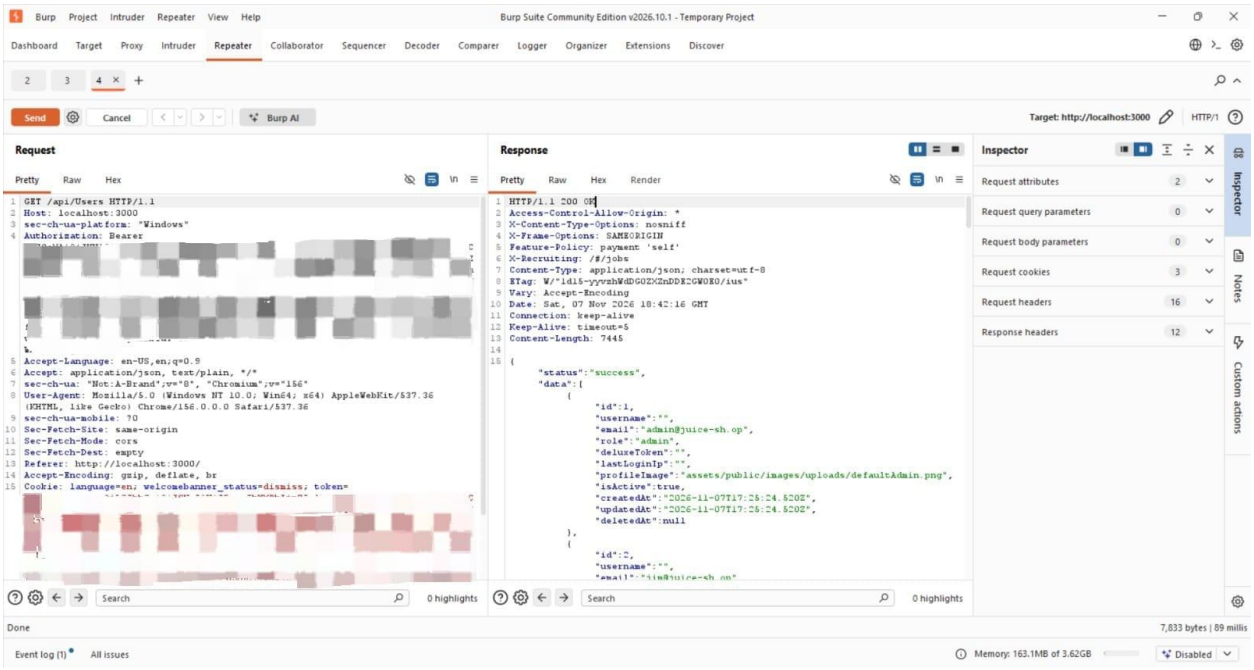


Figure 2: User information exposed through the API endpoint.

Recommendation:

The application should check user permissions on the server side for every request. Users should only be able to access their own information.

7.2 A02 – Security Misconfiguration

Risk Level: Medium to High

During testing, some security configuration weaknesses were identified.

The `/ftp/` directory allowed directory listing. A confidential document named `acquisitions.md` was also publicly accessible.

The file `incident-support.kdbx` could be downloaded. However, its contents were not opened or analyzed. Therefore, the actual sensitivity of its contents was not confirmed.

Some security headers were also missing, including:

- Content-Security-Policy
- Referrer-Policy
- Permissions-Policy

Wildcard CORS (`Access-Control-Allow-Origin: *`) was observed. However, credentialed cross-origin access was not confirmed.

Some backup files returned 403 Forbidden responses.

Finding 1: Public Directory Listing

Status: Confirmed
Risk Level: Medium

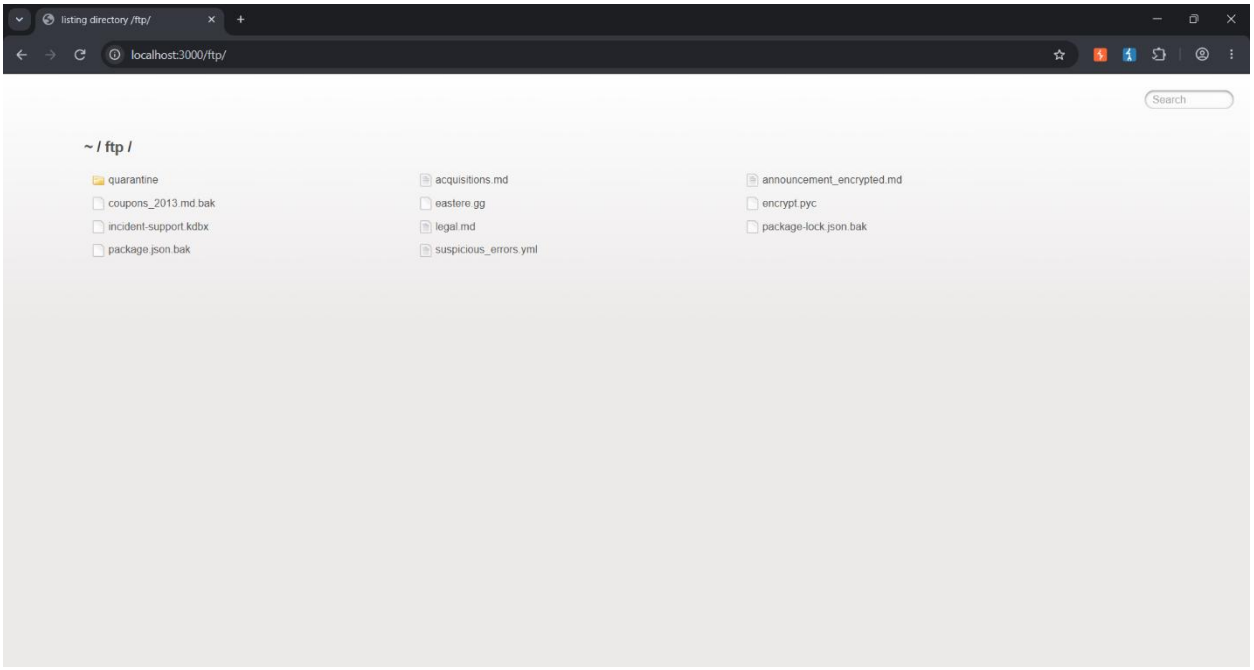


Figure 3: Public directory listing in the application.

Finding 2: Public Access to Confidential Document

Status: Confirmed
Risk Level: Medium

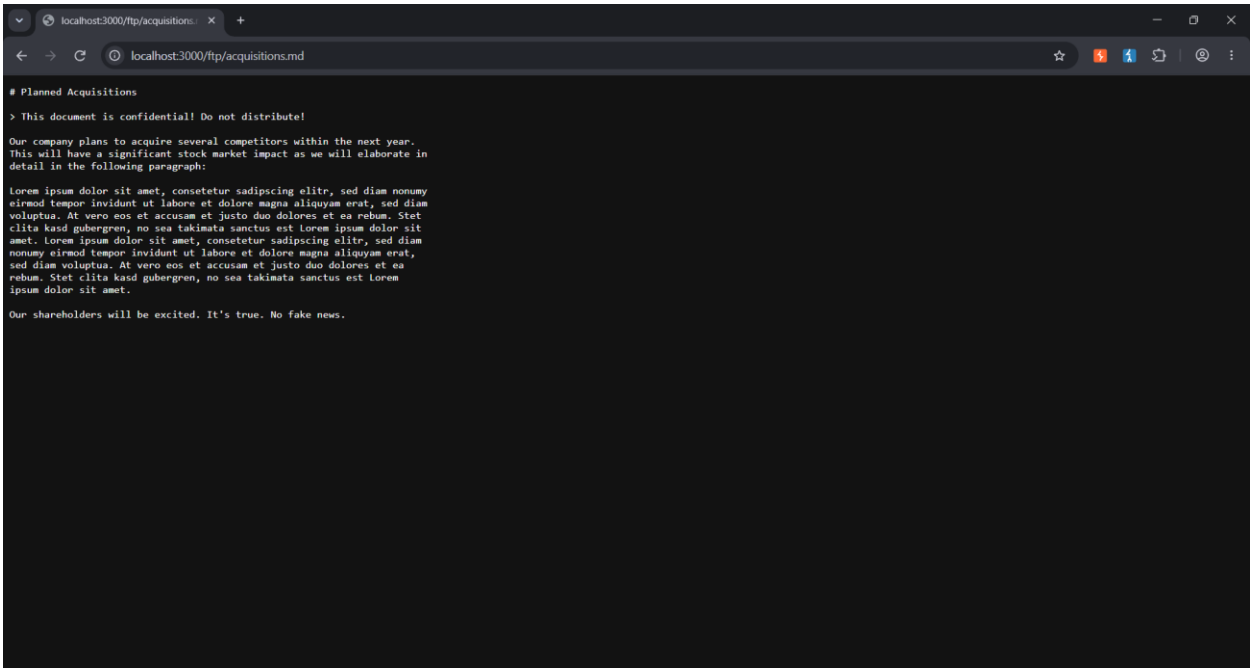


Figure 4: Publicly accessible acquisitions.md file.

Finding 3: Downloadable KeePass File

Status: Download confirmed; contents not analyzed.
Risk Level: High – Preliminary

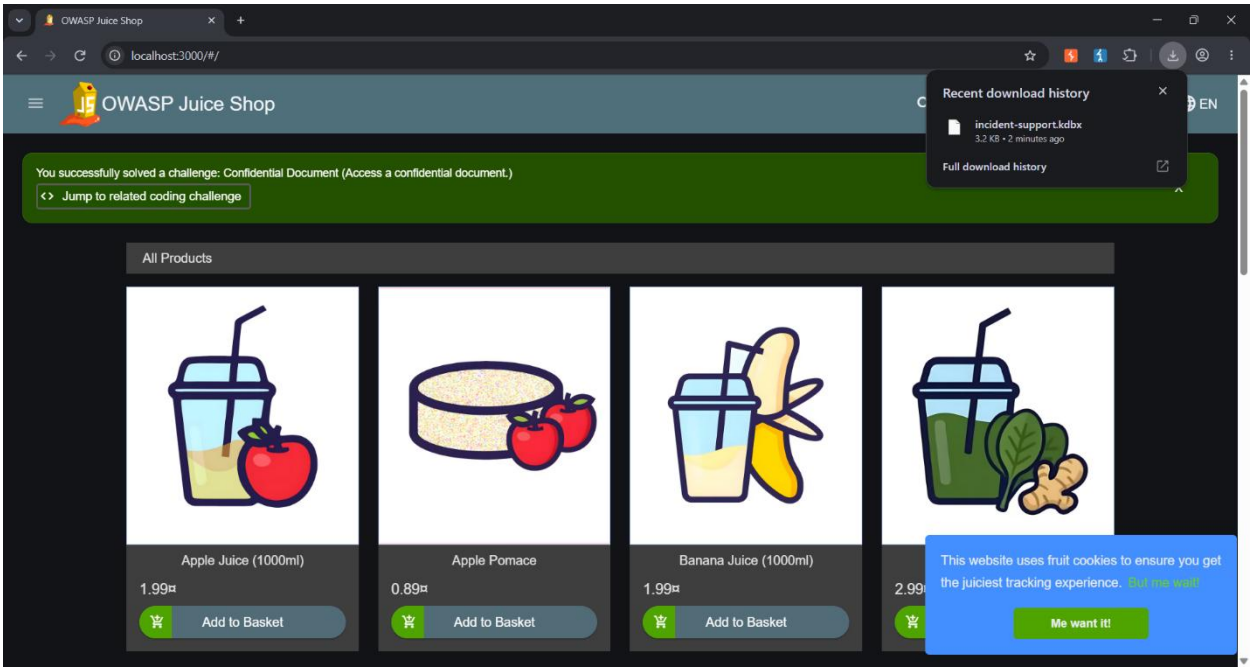


Figure 5: Downloadable incident-support.kdbx file.

Recommendation:

Disable unnecessary directory listing and remove confidential files from public folders.

Configure appropriate security headers and review CORS settings.

7.3 A03 – Software Supply Chain Failures

Risk Level: Not Confirmed

During this assessment, some common dependency and package file locations were checked.

The tested package files were not publicly exposed. A backup lockfile request was also blocked.

However, a complete dependency vulnerability audit was not performed.

Therefore, no confirmed vulnerability was identified under this category.

Recommendation:

The application should regularly check third-party dependencies and update vulnerable packages.

Only trusted software packages should be used.

7.4 A04 – Cryptographic Failures

Risk Level: Not Confirmed

During testing, a login request using local HTTP was observed.

However, this was a local testing environment. This observation alone does not prove that credentials are exposed over an external network.

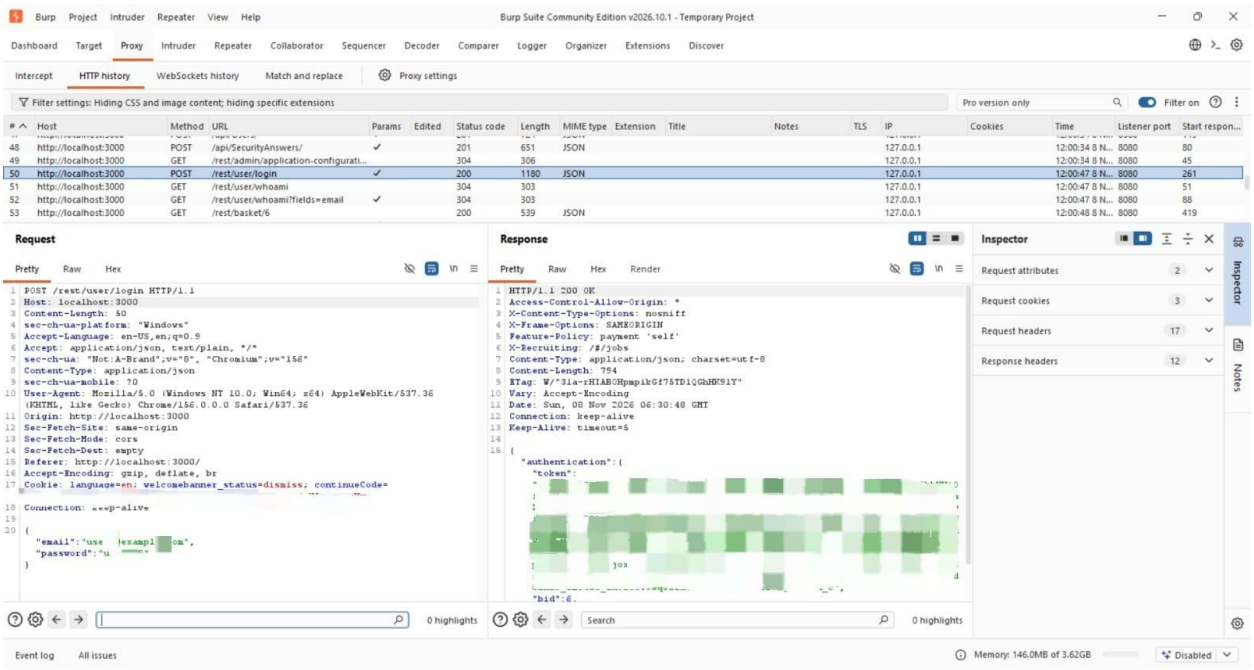


Figure 6: Login request observed over HTTP in the local testing environment.

The `whoami` endpoint returned basic user information, but no password or secret was exposed during the tested request.

Password storage and cryptographic implementation were not fully assessed.

Therefore, no confirmed cryptographic vulnerability was identified.

Recommendation:

Use HTTPS in production environments.

Sensitive information should be protected using strong encryption and secure password hashing methods.

7.5 A05 – Injection

Risk Level: High – Preliminary

During testing, some input fields returned SQL-related error messages.

The product search function displayed database query details and an SQLite syntax error.

A single quote entered into the login field also caused a SQL error. Modified input returned an HTTP 200 response containing an Admin email address.

However, an authenticated admin session was not verified. Therefore, a complete authentication bypass was not confirmed.

User-controlled HTML content also appeared in the review DOM. However, JavaScript execution was not confirmed.

No reflected XSS was confirmed during the tested scenarios.

NoSQL injection was not confirmed. Command injection was not assessed.

Finding 1: SQL Error Information Disclosure

Status: SQL injection indicator observed; exploitability not fully confirmed.
Risk Level: High – Preliminary

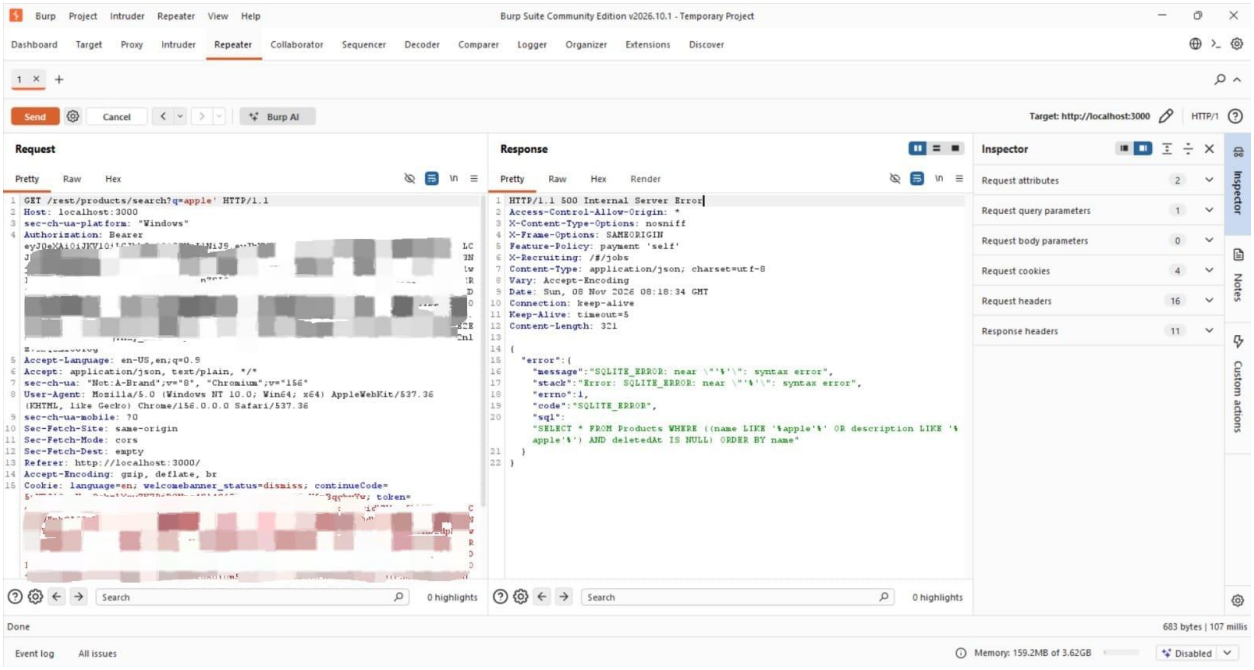


Figure 7: SQL-related error message displayed by the application.

Finding 2: Potential Login Injection

Status: Potential issue; authentication bypass not confirmed.
Risk Level: High – Preliminary

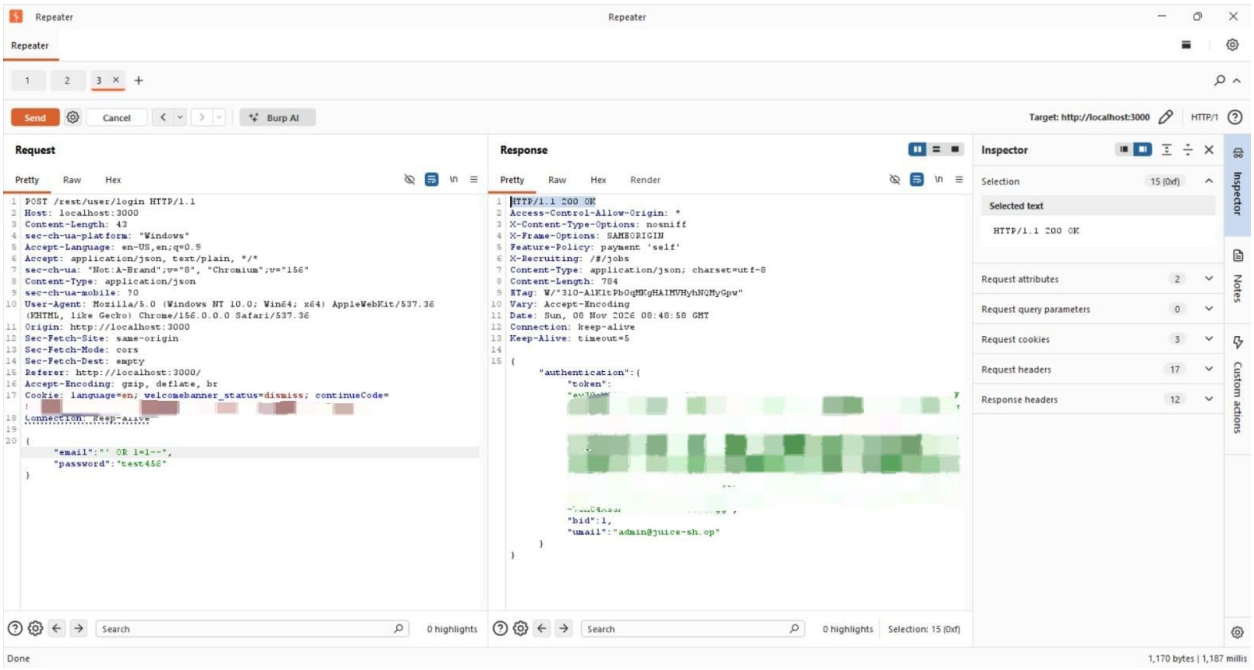


Figure 8: Login input causing an SQL-related response.

Recommendation:

Use parameterized SQL queries and prepared statements.

Validate user inputs on the server side.

Avoid displaying database errors or internal query details to users.

7.6 A06 – Insecure Design

Risk Level: Not Confirmed

During testing, the normal basket function was reviewed.

A duplicate basket request caused a database constraint error.

The checkout process required an address and delivery information. Testing was stopped before completing the payment process.

Five search requests were tested, but no visible rate-limiting behaviour was observed.

An invalid coupon was rejected, and input length validation worked in the tested scenario.

Payment security and server-side pricing validation were not fully tested.

Therefore, no confirmed insecure design vulnerability was identified.

Recommendation:

The application should include proper business logic validation, secure checkout controls, and rate limiting where necessary.

Important business rules should always be checked on the server side.

7.7 A07 – Authentication Failures

Risk Level: Medium

During testing, a five-character password was accepted by the application.

Six failed login attempts were also performed. No visible CAPTCHA or account lockout was observed.

However, backend rate limiting was not verified.

Another important observation was that an old JWT token was still accepted after logout and returned profile information.

This indicates a possible weakness in session invalidation.

Finding 1: Weak Password Policy

Status: Confirmed
Risk Level: Low to Medium

Finding 2: JWT Still Accepted After Logout

Status: Confirmed in the tested scenario
Risk Level: Medium

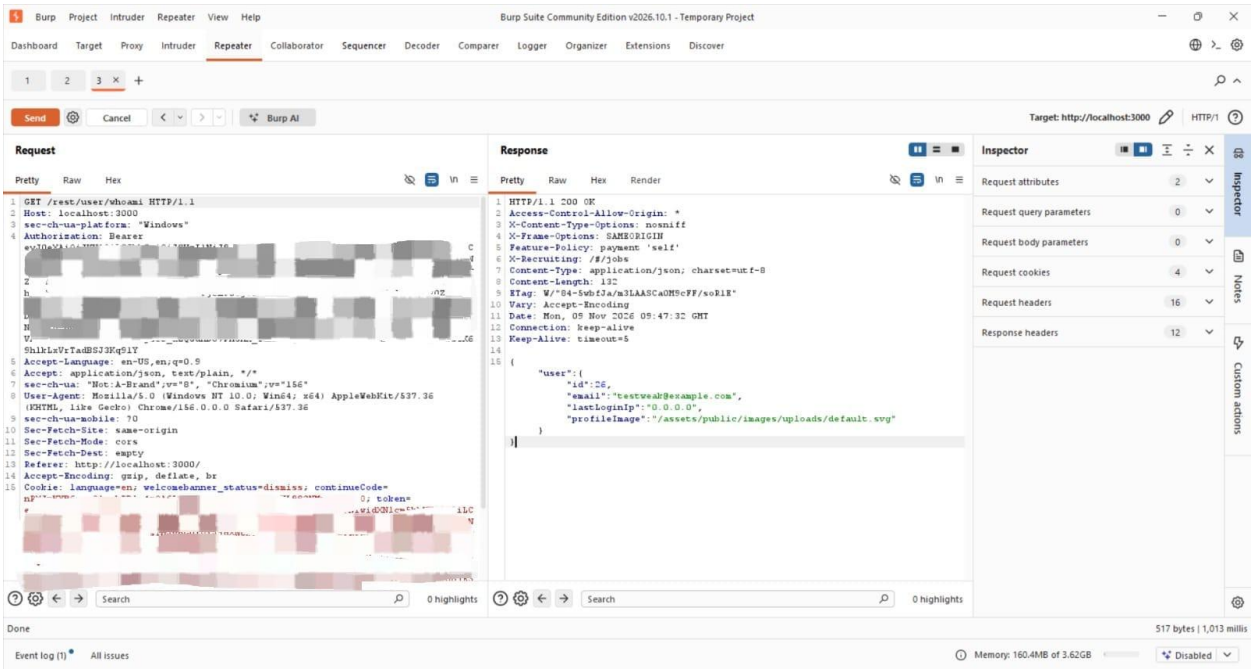


Figure 9: Old JWT token accepted after logout.

Recommendation:

Enforce stronger password requirements.

Implement appropriate login rate limiting and account protection.

Invalidate or revoke active sessions and tokens after logout.

7.8 A08 – Software or Data Integrity Failures

Risk Level: Not Confirmed

During testing, an external JavaScript file without Subresource Integrity (SRI) was observed.

The package lockfile contained integrity entries for dependencies.

However, insecure deserialization, upload integrity, and software update verification were not fully tested.

Therefore, no confirmed integrity vulnerability was identified.

Recommendation:

Use trusted software sources and verify software updates.

Implement file integrity checks where necessary.

Use Subresource Integrity for external scripts when appropriate.

7.9 A09 – Security Logging and Alerting Failures

Risk Level: Not Confirmed

During testing, failed and successful login activities were observed.

Docker logs did not show clear security events for login activities.

A database error message was also found in the logs.

However, historical logs were not available, and alerting behaviour could not be fully verified.

Therefore, a security logging vulnerability was not confirmed.

Recommendation:

Record important security events such as failed logins, successful logins, and unauthorized access attempts.

Monitor application logs and configure alerts for suspicious activities.

Avoid storing sensitive information in logs.

7.10 A10 – Mishandling of Exceptional Conditions

Risk Level: Medium – Observation

During testing, some application errors were observed.

SQL-related error details were displayed during the injection tests.

A duplicate basket request returned an HTTP 500 response with a Sequelize database constraint error.

Other tests, including invalid searches, empty reviews, and longer search inputs, did not crash the application.

The main application continued running despite an optional AI service warning.

These observations show that some exceptional conditions were not handled with user-friendly responses.

Finding: Error Information Disclosure

Status: Observed
Risk Level: Medium – Observation

Recommendation:

Use safe and general error messages for users.

Do not expose database details, internal queries, or technical information.

Handle unexpected errors properly without affecting the application.

---

8. SUMMARY OF RISK RATINGS

The following table summarizes the main findings identified during the assessment.

No.	Finding	Risk Level	Status
1	SQL Injection Indicators	High – Preliminary	Potential
2	Downloadable KeePass File	High – Preliminary	Download confirmed; contents not analyzed
3	User Information Exposure	Medium	Confirmed
4	IDOR – Basket Access	Medium	Confirmed
5	JWT Accepted After Logout	Medium	Confirmed
6	Public Confidential Document	Medium	Confirmed
7	Public Directory Listing	Medium	Confirmed
8	Weak Password Policy	Low–Medium	Confirmed
9	Technical Error Information Disclosure	Medium	Observed

Note: Risk ratings are preliminary and based on the testing performed in this local environment. Potential findings require further verification.

9. RECOMMENDATIONS

Based on the assessment results, the following recommendations are suggested:

1. Implement proper server-side access control for all user requests.
2. Prevent users from accessing other users' private information.
3. Disable unnecessary directory listing.
4. Remove confidential documents and database-related files from public folders.
5. Use parameterized SQL queries to prevent SQL injection.
6. Avoid displaying technical error details to users.
7. Enforce stronger password policies.
8. Implement login rate limiting and appropriate authentication protections.
9. Invalidate active sessions and tokens after logout.
10. Configure suitable security headers and review CORS settings.
11. Regularly update third-party dependencies.
12. Use HTTPS to protect sensitive communication.
13. Improve application logging and monitoring.
14. Validate important application operations on the server side.

10. LIMITATIONS

This assessment was performed in a local lab environment.

There were some limitations during testing:

- Not all application features and endpoints were tested.
- Some potential vulnerabilities could not be fully confirmed.
- A complete dependency audit was not performed.
- Password storage and cryptographic implementation were not fully assessed.
- Payment functionality was not tested completely.
- Historical security logs were not available.
- Some tests were limited to browser and Burp Suite observations.
- No production environment was tested.

Therefore, the findings should not be considered a complete security audit of the application.

11. CONCLUSION

This security assessment helped me understand common web application security weaknesses and the importance of secure application development.

By using OWASP Juice Shop and tools such as Burp Suite, I gained practical experience in reviewing HTTP requests and responses, testing access control, checking authentication behaviour, and identifying possible security issues.

The assessment identified several confirmed security weaknesses and some potential issues that need further investigation.

I also learned how to collect evidence, assign preliminary risk levels, and provide recommendations for improving application security.

Overall, this project improved my practical knowledge of web application security and helped me understand the importance of testing applications before they are deployed.

DECLARATION

I confirm that this security assessment was performed in an authorized local lab environment for educational purposes as part of the SWYNEX Cybersecurity Internship.

The findings and recommendations presented in this report are based on the tests performed during the assessment.

Submitted by:
Fathima Ifadha

Internship:
SWYNEX Cybersecurity Internship

Date:
November 2026